國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab06

姓名：李品融

學號：113310217

Lab06-Q1
這題在練習 Queue 結構的應用，使用 Queue 結構處理發牌，建立自定義佇列類別，並實現牌組初始化。並實現 Ace 進階規則判定與玩家/莊家回合邏輯的功能開發。

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm> // 提供 swap 函式
#include <random> // 提供隨機數功能
#include <ctime> //提供 time() 作為隨機種子

using namespace std;

// 玩家基本資訊
struct Player {// 玩家結構體，包含名稱和分數
 string name; // 玩家名稱
 int score;   // 玩家分數
};

// 卡牌類別
class Card {//建立一個 Card 類別，用來表示單張撲克牌
public:
 string suit; // 花色
 string rank; // 點數
 Card() {} // 預設建構子
 Card(string s, string r) : suit(s), rank(r) {}

 // 顯示卡牌資訊
 void display() const {
  cout << rank << " of " << suit << endl;
 }
};
```

```
class Deck { // 牌組類別，負責生成和洗牌
private:
 Card cards[52]; // 52 張牌
 int index; // 目前發牌索引
public:
 Deck() : index(0) {// 初始化牌組
  string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" };// 四種花色
  string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q",
"K", "A" };// 13 張牌
  int k = 0;
  for (int i = 0; i < 4; i++) { // 4 種花色
   for (int j = 0; j < 13; j++) { // 13 種點數
    cards[k] = Card(suits[i], ranks[j]);
    k++;
   }
  }
 }

 void shuffleDeck() {// 洗牌
  srand(time(0)); // 當前時間作為隨機種子，確保每次洗牌的結果不同
  for (int i = 51; i > 0; i--) {// 從最後一張牌開始，向前遍歷
   int j = rand() % (i + 1);// 生成隨機索引 j
   swap(cards[i], cards[j]);// 交換牌組中的兩張牌
  }
 }

 // 取得指定位置的牌
 Card getCard(int i) const {
  return cards[i];
 }
};
```

```
// Queue 類別
// 先進先出，洗好的牌從尾端進入，發牌從前端取出
class Queue {
private:
 vector<Card> deck; // 用來存放卡牌的容器
 int Front;         // 指向牌堆的前端
 int Rear;          // 指向牌堆的尾端
 int capacity;      // 容器最大容量
public:
 // 建構函式，初始化指標，-1 代表目前是空的
 Queue(int size) : Front(-1), Rear(-1), capacity(size) {}
 // 插入元素到佇列尾端

 // 檢查佇列是否為空
 bool isEmpty() const {
  //提示:front 和 rear 的關係，並且有兩個情況要檢查(用&&)
  // 如果 Front 為-1 或 Front 超過了 Rear，表示裡面沒牌了
  return (Front == -1 || Front > Rear);
 }

 // 檢查佇列是否已滿
 bool isFull() const {
  //提示:deck 的大小和 capacity 的關係
  // 當 Rear 指標到達容量上限-1 時即為滿了
  return (Rear == capacity - 1);
 }

 // 將洗好的牌放入發牌堆
 void enqueue(Card card) {
  if (isFull()) { // 檢查是否滿
   cout << "牌堆已滿，無法插入！" << endl;
   return;
  }
  if (isEmpty()) Front = 0; // 第一張牌進入時，將 Front 設為 0
   deck.push_back(card); // 插入元素到尾端

  Rear++;              // 尾端指標移動
  }
```

```
  // 查看下一張要發的是哪張牌
  Card front() {
   if (isEmpty()) {
    cout << "牌堆為空，無法獲取最前端的牌！" << endl;
    return Card("Empty", "0");
   }
   return deck[Front];
  }

  // 發牌，將 Front 指標後移，表示該牌已不在發牌堆中
  void dequeue() {
   if (isEmpty()) { // 檢查是否空
    cout << "牌堆為空，無法獲取最前端的牌！" << endl;
    return;
   }
   Front++; // 移動前端指標模擬取出的動作
  }
};


// 初始化發牌堆，將 52 張牌洗好後全部放入 Queue 中
void initializeDeck(Queue& cardDeck) {// 初始化並洗牌
 Deck deck;
 cout << "初始牌堆: \n";
 for (int i = 0; i < 52; i++) {
  deck.getCard(i).display(); //印出原始牌堆
 }

 deck.shuffleDeck();//洗牌
 cout << "\n洗牌後的牌堆: \n";
 for (int i = 0; i < 52; i++) {
  deck.getCard(i).display();//印出洗牌後的牌堆
  Card c = deck.getCard(i);
  cardDeck.enqueue(c); // 將洗好的牌放入 Queue
 }
 //提示:將洗好的牌放入牌堆
 //使用for 迴圈，將洗好的牌(deck.getCard(i))放入牌堆
}
```

```
// 初始化，發送起始的兩張牌
void initializePlayer(Player* player, string name, Queue& cardDeck) { // 初始化玩家，發兩張牌
 player->name = name;// 玩家名稱
 player->score = 0;// 玩家分數
 cout << player->name << " 抽到的牌: ";

 for (int i = 0; i < 2; i++) {// 發兩張牌
  //提示:從牌堆中取出一張牌，然後從牌堆中移除這張牌
  //注意:卡牌變數的類型是 Card，並且使用 front 抽牌
  Card drawnCard = cardDeck.front(); // 從 Queue 的前端看牌
  cardDeck.dequeue();              // 從 Queue 移除這張牌
  // 將抽到的牌印出
  cout << drawnCard.rank << " of " << drawnCard.suit << "  ";

  if (drawnCard.rank == "A") {
   //若目前剛好 10 分，A 可當作 11 分並湊成 21 分，否則當作 1 分
   player->score += (player->score == 10) ? 11 : 1; //計算 A 分數
  }

  else if (drawnCard.rank == "J" || drawnCard.rank == "Q" || drawnCard.rank == "K") {
   player->score += 10; // J、Q、K 一律計算 10 分
  }

  else player->score += stoi(drawnCard.rank); //計算 2-10 分數
 }

 cout << "\n";
 cout << player->name << " 玩家的初始分數: " << player->score << endl;//印出玩家的初始分數
}
```

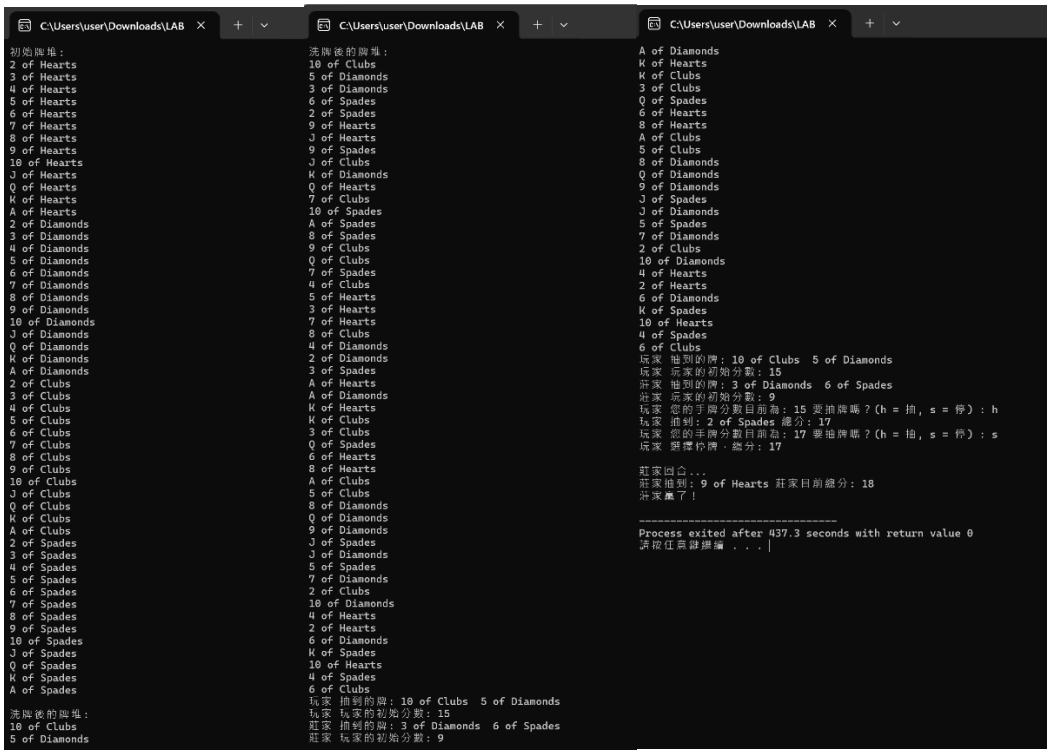
```
// 玩家的回合，處理抽牌或停牌
void playerTurn(Player* player, Queue& cardDeck) { // 玩家回合
 char choice;
 while (player->score < 21 && !cardDeck.isEmpty()) { // 玩家小於 21 點且牌堆不為空
  cout << player->name << " 您的手牌分數目前為: " << player->score << " 要抽牌嗎？
(h = 抽, s = 停) : ";//顯示選擇
  cin >> choice; //輸入選擇

  if (choice == 'h') { //如果玩家選擇抽牌
  //提示:從牌堆中取出一張牌，然後從牌堆中移除這張牌
  //注意:卡牌變數的類型是 Card，並且使用 front 抽牌

   Card newCard = cardDeck.front(); // 宣告變數類型為 Card 並從前端拿牌
   cardDeck.dequeue();              // 從牌堆移除

   if (newCard.rank == "A") {
        //若目前剛好 10 分，A 可當作 11 分並湊成 21 分，否則當作 1 分
    player->score += (player->score == 10) ? 11 : 1; //計算 A 點數
   }

   else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K")
{
    player->score += 10; // J、Q、K 一律計算 10 分
   }

   else player->score += stoi(newCard.rank); //計算 2-10 點數
   //印出抽到的牌和總點數

   cout << player->name << " 抽到: " << newCard.rank << " of " << newCard.suit
<< " 總分: " << player->score << endl;
   if (player->score > 21) { //如果玩家大於 21 點
    cout << player->name << " 爆掉了！遊戲結束。\n"; //玩家爆牌，莊家獲勝
    return;
   }
  }
```

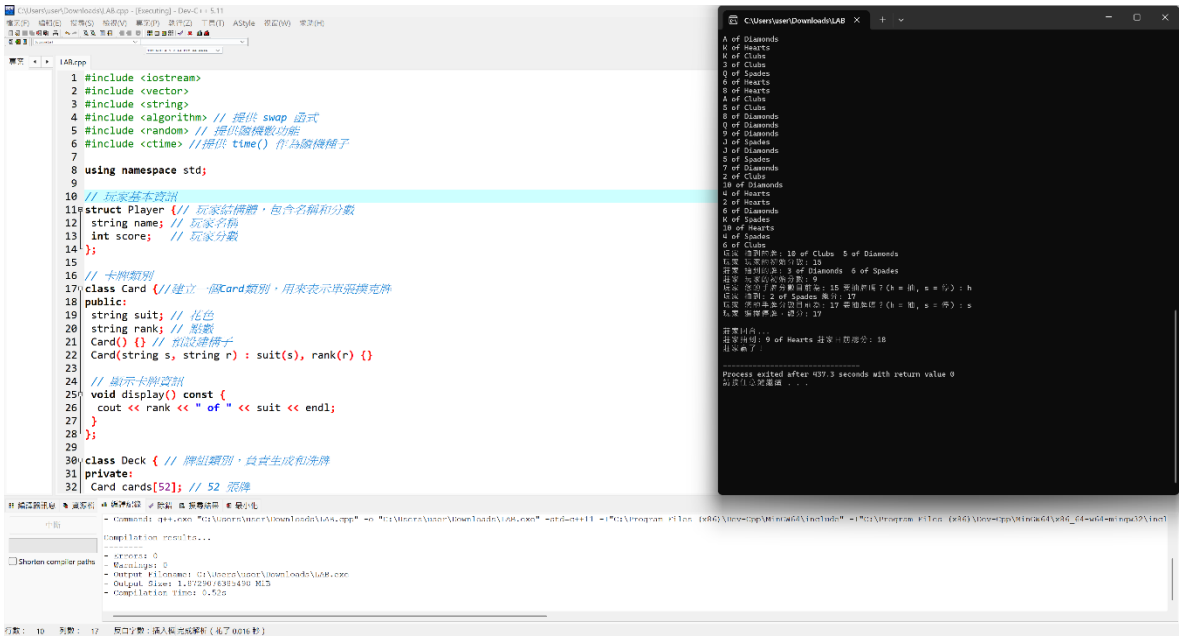
```
  else if (choice == 's') { //如果玩家選擇停牌
   cout << player->name << " 選擇停牌，總分: " << player->score << endl;//印出總
點數
   break;
  }
  else {
   cout << "請輸入有效選項 (h = 抽, s = 停)！" << endl; //輸入無效
  }
 }
}


// 莊家回合
void dealerTurn(Player* dealer, Queue& cardDeck) { // 莊家回合
 while (dealer->score < 17 && !cardDeck.isEmpty())  {// 莊家小於 17 點且牌堆不為空
 //提示:從牌堆中取出一張牌，然後從牌堆中移除這張牌
 //注意:卡牌變數的類型是 Card，並且使用 front 抽牌

  Card newCard = cardDeck.front(); // 宣告變數類型為 Card
  cardDeck.dequeue();              // 移除

  if (newCard.rank == "A") {
   // 若目前剛好 10 分，A 可當作 11 分並湊成 21 分，否則當作 1 分
   dealer->score += (dealer->score == 10) ? 11 : 1; // 計算 A 點數
  }

  else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K") {
   dealer->score += 10; // J、Q、K 一律計算 10 分
  }

  else dealer->score += stoi(newCard.rank); // 計算 2-10 點數
  //印出抽到的牌和總點數

  cout << "莊家抽到: " << newCard.rank << " of " << newCard.suit << " 莊家目前總
分: " << dealer->score << endl;
  if (dealer->score > 21) { // 如果莊家大於 21 點
   cout << "莊家爆了！玩家獲勝！" << endl; // 玩家獲勝
   return;
```

```
  }
 }
}

// 判斷勝負
void determineWinner(Player* player, Player* dealer) {
 if (player->score > 21) cout << player->name << " 爆了！莊家獲勝！\n"; //玩家爆牌，莊家獲勝
 else if (dealer->score > 21 || player->score > dealer->score)
  cout << player->name << " 贏了！\n"; //莊家爆牌或玩家點數大於莊家，玩家獲勝

 else if (player->score == dealer->score) cout << "平手！\n"; //莊家與玩家點數相同，平手
 else cout << "莊家贏了！\n";
}

int main() {
 srand(time(0)); // 確保隨機種子初始化
 Queue cardDeck(52); // 宣告牌堆，建立一個最大容量為 52 的 Queue
 initializeDeck(cardDeck); // 初始化牌堆

 Player player, dealer; // 宣告並初始化莊家以及玩家
 initializePlayer(&player, "玩家", cardDeck);
 initializePlayer(&dealer, "莊家", cardDeck);

 // 玩家回合
 playerTurn(&player, cardDeck);

 // 若玩家沒爆牌，將輪到莊家行動
 if (player.score <= 21) {
  cout << "\n 莊家回合...\n";
  dealerTurn(&dealer, cardDeck);
  determineWinner(&player, &dealer);
 }

 return 0;
}
```

Result

範例一:


C:\Users\user\Downloads\LAB
初始牌堆:
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
J of Spades
Q of Spades
K of Spades
A of Spades
洗牌後的牌堆:
10 of Clubs
5 of Diamonds
C:\Users\user\Downloads\LAB
洗牌後的牌堆:
10 of Clubs
5 of Diamonds
3 of Diamonds
6 of Spades
2 of Spades
9 of Hearts
J of Hearts
9 of Spades
J of Clubs
K of Diamonds
Q of Hearts
7 of Clubs
10 of Spades
A of Spades
8 of Spades
9 of Clubs
Q of Clubs
7 of Spades
4 of Clubs
5 of Hearts
3 of Hearts
7 of Hearts
8 of Clubs
4 of Diamonds
2 of Diamonds
3 of Spades
A of Hearts
A of Diamonds
K of Hearts
K of Clubs
3 of Clubs
Q of Spades
6 of Hearts
8 of Hearts
A of Clubs
5 of Clubs
8 of Diamonds
Q of Diamonds
9 of Diamonds
J of Spades
J of Diamonds
5 of Spades
7 of Diamonds
2 of Clubs
10 of Diamonds
4 of Hearts
2 of Hearts
6 of Diamonds
K of Spades
10 of Hearts
4 of Spades
6 of Clubs
玩家 抽到的牌: 10 of Clubs 5 of Diamonds
玩家 玩家的初始分數: 15
莊家 抽到的牌: 3 of Diamonds 6 of Spades
莊家 玩家的初始分數: 9
C:\Users\user\Downloads\LAB
A of Diamonds
K of Hearts
K of Clubs
3 of Clubs
Q of Spades
6 of Hearts
8 of Hearts
A of Clubs
5 of Clubs
8 of Diamonds
Q of Diamonds
9 of Diamonds
J of Spades
J of Diamonds
5 of Spades
7 of Diamonds
2 of Clubs
10 of Diamonds
4 of Hearts
2 of Hearts
6 of Diamonds
K of Spades
10 of Hearts
4 of Spades
6 of Clubs
玩家 抽到的牌: 10 of Clubs 5 of Diamonds
玩家 玩家的初始分數: 15
莊家 抽到的牌: 3 of Diamonds 6 of Spades
莊家 玩家的初始分數: 9
玩家 您的手牌分數目前為: 15 要抽牌嗎?(h = 抽, s = 停) : h
玩家 抽到: 2 of Spades 總分: 17
玩家 您的手牌分數目前為: 17 要抽牌嗎?(h = 抽, s = 停) : s
玩家 選擇停牌,總分: 17
莊家回合...
莊家抽到: 9 of Hearts 莊家目前總分: 18
莊家贏了!
Process exited after 437.3 seconds with return value 0
請按任意鍵繼續 . . .


C:\Users\user\Downloads\LAB.cpp - [Executing] - Dev-C++ 5.11
檔案(F) 編輯(E) 搜尋(S) 檢視(V) 專案(P) 執行(E) 工具(T) AStyle 視窗(W) 說明(H)
LAB.cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm> // 提供 swap 函式
#include <random> // 提供隨機數功能
#include <ctime> //提供 time() 作為隨機種子
using namespace std;
// 玩家基本資訊
struct Player {// 玩家結構體,包含名稱和分數
string name; // 玩家名稱
int score; // 玩家分數
};
// 卡牌類別
class Card {//建立一個Card類別,用來表示單張撲克牌
public:
string suit; // 花色
string rank; // 點數
Card() {} // 預設建構子
Card(string s, string r) : suit(s), rank(r) {}
// 顯示卡牌資訊
void display() const {
cout << rank << " of " << suit << endl;
}
};
class Deck { // 牌組類別,負責生成和洗牌
private:
Card cards[52]; // 52 張牌
C:\Users\user\Downloads\LAB
A of Diamonds
K of Hearts
K of Clubs
3 of Clubs
Q of Spades
6 of Hearts
8 of Hearts
A of Clubs
5 of Clubs
8 of Diamonds
Q of Diamonds
9 of Diamonds
J of Spades
J of Diamonds
5 of Spades
7 of Diamonds
2 of Clubs
10 of Diamonds
4 of Hearts
2 of Hearts
6 of Diamonds
K of Spades
10 of Hearts
4 of Spades
6 of Clubs
玩家 抽到的牌: 10 of Clubs 5 of Diamonds
玩家 玩家的初始分數: 15
莊家 抽到的牌: 3 of Diamonds 6 of Spades
莊家 玩家的初始分數: 9
玩家 您的手牌分數目前為: 15 要抽牌嗎?(h = 抽, s = 停) : h
玩家 抽到: 2 of Spades 總分: 17
玩家 您的手牌分數目前為: 17 要抽牌嗎?(h = 抽, s = 停) : s
玩家 選擇停牌,總分: 17
莊家回合...
莊家抽到: 9 of Hearts 莊家目前總分: 18
莊家贏了!
Process exited after 437.3 seconds with return value 0
請按任意鍵繼續 . . .
Compilation results...
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\user\Downloads\LAB.exe
- Output Size: 1.87290 (438x496 MLS
- Compilation Time: 0.52s
行數: 10 列數: 17 反白分數:滿人模式統計(花了 0.016 秒)

範例二:

```
C:\Users\user\Downloads\LAB
初始牌組：
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
J of Spades
Q of Spades
K of Spades
A of Spades

洗牌後的牌組：
3 of Spades
```

```
C:\Users\user\Downloads\LAB
洗牌後的牌組：
3 of Spades
J of Hearts
10 of Clubs
J of Diamonds
2 of Spades
8 of Hearts
6 of Hearts
7 of Diamonds
4 of Diamonds
5 of Diamonds
9 of Hearts
Q of Diamonds
K of Spades
3 of Clubs
Q of Spades
3 of Hearts
Q of Hearts
7 of Hearts
A of Clubs
7 of Clubs
10 of Diamonds
2 of Clubs
9 of Diamonds
4 of Clubs
2 of Clubs
7 of Spades
5 of Hearts
K of Hearts
3 of Diamonds
A of Hearts
2 of Hearts
4 of Spades
10 of Spades
J of Spades
9 of Spades
10 of Hearts
8 of Diamonds
J of Clubs
2 of Diamonds
4 of Hearts
K of Clubs
9 of Clubs
5 of Spades
A of Spades
6 of Diamonds
A of Diamonds
5 of Clubs
6 of Clubs
8 of Clubs
K of Diamonds
8 of Spades
6 of Spades
玩家 抽到的牌：3 of Spades  J of Hearts
玩家 玩家的初始分數：13
莊家 抽到的牌：10 of Clubs  J of Diamonds
```

```
C:\Users\user\Downloads\LAB
K of Hearts
3 of Diamonds
A of Hearts
2 of Hearts
4 of Spades
10 of Spades
J of Spades
9 of Spades
10 of Hearts
8 of Diamonds
J of Clubs
2 of Diamonds
4 of Hearts
K of Clubs
9 of Clubs
5 of Spades
A of Spades
6 of Diamonds
A of Diamonds
5 of Clubs
6 of Clubs
8 of Clubs
K of Diamonds
8 of Spades
6 of Spades
玩家 抽到的牌：3 of Spades  J of Hearts
玩家 玩家的初始分數：13
莊家 抽到的牌：10 of Clubs  J of Diamonds
莊家 玩家的初始分數：20
玩家 您的手牌分數目前為：13 要抽牌嗎？(h = 抽, s = 停)：h
玩家 抽到：2 of Spades 總分：15
玩家 您的手牌分數目前為：15 要抽牌嗎？(h = 抽, s = 停)：h
玩家 抽到：8 of Hearts 總分：23
玩家 爆掉了！遊戲結束。

--------------------------------
Process exited after 73.62 seconds with return value 0
請按任意鍵繼續 . . .
```

```
C:\Users\user\Downloads\LAB.cpp - [Executing] - Dev-C++ 5.11
檔案(F) 編輯(E) 搜尋(S) 檢視(V) 專案(P) 執行(E) 工具(T) AStyle 視窗(W) 求助(H)
LAB.cpp
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm> // 提供 swap 格式
5  #include <random> // 提供隨機數功能
6  #include <ctime> //提供 time() 作為隨機種子
7
8  using namespace std;
9
10 // 玩家基本資訊
11 struct Player {// 玩家結構體，包含名稱和分數
12  string name; // 玩家名稱
13  int score;   // 玩家分數
14 };
15
16 // 卡牌類別
17 class Card {//建立一個Card類別，用來表示單張撲克牌
18 public:
19  string suit; // 花色
20  string rank; // 點數
21  Card() {} // 預設建構子
22  Card(string s, string r) : suit(s), rank(r) {}
23
24  // 顯示卡牌資訊
25  void display() const {
26   cout << rank << " of " << suit << endl;
27  }
28 };
29
30 class Deck { // 牌組類別，負責生成和洗牌
31 private:
32  Card cards[52]; // 52 張牌
```

```
C:\Users\user\Downloads\LAB
K of Hearts
3 of Diamonds
A of Hearts
2 of Hearts
4 of Spades
10 of Spades
J of Spades
9 of Spades
10 of Hearts
8 of Diamonds
J of Clubs
2 of Diamonds
4 of Hearts
K of Clubs
9 of Clubs
5 of Spades
A of Spades
6 of Diamonds
A of Diamonds
5 of Clubs
6 of Clubs
8 of Clubs
K of Diamonds
8 of Spades
6 of Spades
玩家 抽到的牌：3 of Spades  J of Hearts
玩家 玩家的初始分數：13
莊家 抽到的牌：10 of Clubs  J of Diamonds
莊家 玩家的初始分數：20
玩家 您的手牌分數目前為：13 要抽牌嗎？(h = 抽, s = 停)：h
玩家 抽到：2 of Spades 總分：15
玩家 您的手牌分數目前為：15 要抽牌嗎？(h = 抽, s = 停)：h
玩家 抽到：8 of Hearts 總分：23
玩家 爆掉了！遊戲結束。

--------------------------------
Process exited after 73.62 seconds with return value 0
請按任意鍵繼續 . . .
```

```
編譯訊息  資源檔  搜尋結果  除錯  檢查結果  高小白
一般
shorten compiler paths
- Command: g++.exe "C:\Users\user\Downloads\LAB.cpp" -o "C:\Users\user\Downloads\LAB.exe" -std=c++11 -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\include" -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\x86_64-w64-mingw32\inc

Compilation results...
--------
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\user\Downloads\LAB.exe
- Output Size: 1.0720796585398 MiB
- Compilation Time: 0.32s
行數： 250  列數： 2  展開次數：插入模板或解析 (花了 0.359 秒)
```

範例三:

```
C:\Users\user\Downloads\LAB
洗牌後的牌堆：
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
J of Spades
Q of Spades
K of Spades
A of Spades

洗牌後的牌堆：
3 of Hearts
```

```
C:\Users\user\Downloads\LAB
洗牌後的牌堆：
3 of Hearts
6 of Hearts
5 of Diamonds
8 of Spades
9 of Clubs
J of Spades
2 of Diamonds
J of Diamonds
8 of Clubs
Q of Spades
10 of Spades
7 of Hearts
A of Clubs
7 of Diamonds
K of Spades
4 of Diamonds
3 of Diamonds
A of Diamonds
K of Diamonds
J of Hearts
A of Spades
K of Hearts
6 of Clubs
2 of Spades
5 of Clubs
9 of Hearts
9 of Spades
Q of Clubs
2 of Hearts
K of Clubs
9 of Diamonds
3 of Spades
4 of Hearts
4 of Spades
2 of Clubs
8 of Diamonds
5 of Spades
4 of Clubs
8 of Hearts
7 of Clubs
Q of Diamonds
Q of Hearts
7 of Spades
10 of Diamonds
3 of Clubs
A of Hearts
6 of Spades
10 of Clubs
6 of Diamonds
J of Clubs
5 of Hearts
10 of Hearts
玩家 抽到的牌堆：3 of Hearts  6 of Hearts
玩家 玩家的初始分數：9
莊家 抽到的牌堆：5 of Diamonds  8 of Spades
```

```
C:\Users\user\Downloads\LAB
Q of Clubs
2 of Hearts
K of Clubs
9 of Diamonds
3 of Spades
4 of Hearts
4 of Spades
2 of Clubs
8 of Diamonds
5 of Spades
4 of Clubs
8 of Hearts
7 of Clubs
Q of Diamonds
Q of Hearts
7 of Spades
10 of Diamonds
3 of Clubs
A of Hearts
6 of Spades
10 of Clubs
6 of Diamonds
J of Clubs
5 of Hearts
10 of Hearts
玩家 抽到的牌堆：3 of Hearts  6 of Hearts
玩家 玩家的初始分數：9
莊家 抽到的牌堆：5 of Diamonds  8 of Spades
莊家 莊家的初始分數：13
玩家 玩家的牌分數目前為：9 要抽牌嗎？(h = 抽, s = 停) : h
玩家 抽到：9 of Clubs 總分：18
玩家 您的牌分數目前為：18 要抽牌嗎？(h = 抽, s = 停) : s
玩家 選擇停牌，總分：18

莊家回合...
莊家抽到：J of Spades 莊家目前總分：23
莊家爆了！玩家獲勝！
玩家 贏了！

--------------------------------
Process exited after 69.5 seconds with return value 0
請按任意鍵繼續 . . .
```

```
C:\Users\user\Downloads\LAB.cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm> // 提供 swap 函式
#include <random> // 提供隨機數功能
#include <ctime> //提供 time() 作為隨機種子

using namespace std;

// 玩家基本資訊
struct Player {// 玩家結構體，包含名稱和分數
  string name; // 玩家名稱
  int score;   // 玩家分數
};

// 卡牌類別
class Card {//建立一個Card類別，用來表示單張撲克牌
public:
  string suit; // 花色
  string rank; // 點數
  Card() {} // 預設建構子
  Card(string s, string r) : suit(s), rank(r) {}

  // 顯示卡牌資訊
  void display() const {
    cout << rank << " of " << suit << endl;
  }
};

class Deck { // 牌組類別，負責生成和洗牌
private:
  Card cards[52]; // 52 張牌
```

```
C:\Users\user\Downloads\LAB
Q of Clubs
2 of Hearts
K of Clubs
9 of Diamonds
3 of Spades
4 of Hearts
4 of Spades
2 of Clubs
8 of Diamonds
5 of Spades
4 of Clubs
8 of Hearts
7 of Clubs
Q of Diamonds
Q of Hearts
7 of Spades
10 of Diamonds
3 of Clubs
A of Hearts
6 of Spades
10 of Clubs
6 of Diamonds
J of Clubs
5 of Hearts
10 of Hearts
玩家 抽到的牌堆：3 of Hearts  6 of Hearts
玩家 玩家的初始分數：9
莊家 抽到的牌堆：5 of Diamonds  8 of Spades
莊家 莊家的初始分數：13
玩家 玩家的牌分數目前為：9 要抽牌嗎？(h = 抽, s = 停) : h
玩家 抽到：9 of Clubs 總分：18
玩家 您的牌分數目前為：18 要抽牌嗎？(h = 抽, s = 停) : s
玩家 選擇停牌，總分：18

莊家回合...
莊家抽到：J of Spades 莊家目前總分：23
莊家爆了！玩家獲勝！
玩家 贏了！

--------------------------------
Process exited after 69.5 seconds with return value 0
請按任意鍵繼續 . . .
```

Lab06-Q3
討論 Queue 與 Stack 在執行發牌上的差異。
Discussion Section
若使用 Queue (佇列)，其先進先出的特性完美模擬了真實的發牌情境，洗好的牌依序疊入後，發牌時從牌堆的頂端取出，確保了發牌順序與洗牌後的原始排列一致，非常符合現實物理規律! 若採用 Stack (堆疊)，其後進先出的特性會導致發牌順序與洗牌後的放入順序完全相反，最後一張洗好的牌會最先被發出。在程式製作與邏輯上，Queue 更能對應排隊後發的概念。